ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Designing higher-order functions / Part II

We have illustrated the power of higher-order definitions as a tool to *factor out* the common patterns of collection processing. In this lecture we look at further features of higher-order functions and their types.

Paradigmatic higher-order schemes

Currying and uncurrying. Functions of more than one argument may be defined in two different ways: by passing all arguments at once (in a tuple), or by passing them one by one in a sequence. In Haskell:

```
add  :: (Int, Int) -> Int       -- uncurried: takes a pair
add (x, y) = x + y

addc :: Int -> Int -> Int       -- curried: takes arguments one by one
addc x y = x + y
```

The two definitions are isomorphic: we can transform one into the other and vice versa. The functions that witness this isomorphism are `curry` and `uncurry`:

```
curry   :: ((a, b) -> c) -> a -> b -> c
curry f x y = f (x, y)

uncurry :: (a -> b -> c) -> (a, b) -> c
uncurry f (x, y) = f x y
```

Then:

```
add  = uncurry addc
addc = curry add
```

In Haskell, all functions are curried by default — writing `addc x y = x + y` already gives a function of type `Int -> Int -> Int`, and partial application is always available: `addc 1` is a function `Int -> Int`. In Scala, functions can be defined in either curried or uncurried style.

```
def add(x: Int, y: Int): Int = x + y        // default: uncurried
def addc(x: Int)(y: Int): Int = x + y       // explicit currying via multiple param lists
```

If defined in curried style, partial application works the same way: `addc(1)` returns a function `Int => Int`. The difference is that Scala's default is uncurried (`def add(x: Int, y: Int)`), in which case partial application requires the placeholder syntax `add(1, _)` discussed in more detail below.

Operator sections. Operator sections are idiomatic in Haskell: simply put they are partially applied operators, written in parentheses with one argument supplied. In Haskell, any infix operator can be sectioned this way:

```
(* 2)     -- the function that multiplies its argument by 2
(+ 1)     -- the function that adds 1 to its argument
(> 5)     -- the function that tests whether its argument is greater than 5
(++ "!")  -- the function that appends "!" to a string
(1 :)     -- the function that conses 1 onto a list
```

You can section on either side: `(10 /)` is "divide 10 by" (i.e. `\x -> 10 / x`), while `(/ 10)` is "divide by 10" (i.e. `\x -> x / 10`). In each case, the section is equivalent to a lambda, but more concise. Sections are first-class values: they can be bound to names, passed as arguments, composed with other functions.

Sections are particularly useful as arguments to higher-order functions. We have seen examples of this use:

```
map (* 2) [1,2,3]        -- [2,4,6]
filter (> 3) [1,2,3,4,5] -- [4,5]
foldr (+) 0 [1,2,3]      -- 6
```

Without sections, the same code would require lambdas: `map (\x -> x * 2) [1,2,3]`, etc.

Scala note. Scala has no direct equivalent of sections. The `_` placeholder looks similar:

```
List(1,2,3).map(_ * 2)        // [2,4,6]
List(1,2,3).filter(_ > 3)     // []
```

However, that is a different mechanism: `_ * 2` is syntactic sugar for the lambda `x => x * 2`, and it only works *inline*, inside an expression where the compiler can infer the type of `_`. You cannot write `val double = _ * 2` as a standalone definition — Scala's type inference needs a context. By contrast, Haskell's `(* 2)` is a self-contained value of type `Num a => a -> a` that can be used anywhere a function is expected.

Composing functions. Function composition can be defined as a higher-order function that takes two functions and returns a new function applying the first, then the second:

```
compose :: (a -> b) -> (b -> c) -> (a -> c)
compose f g = \x -> g (f x)
```

Haskell provides this as the built-in operator `(.)`, with the conventional mathematical ordering (second function first):

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
(g . f) x = g (f x)
```

So `(+1) . (*2)` is a function that doubles, then adds one. Function composition is one of the most characteristic idioms of Haskell programming: instead of writing `\x -> f (g (h x))`, one writes `f . g . h`.

Elixir note. Elixir provides the **pipe operator** `|>` as an alternative to composition. Where Haskell writes `f . g . h $ x`, or equivalently (apply `h`, then `g`, then `f` to `x`) Elixir writes:

```
x |> h.()
  |> g.()
  |> f.()
```

The pipe passes the result of the left-hand side as the first argument of the right-hand side. The data flows left-to-right, which many find more readable for chains of transformations. The pipe is not function composition (it applies immediately rather than building a new function), but it serves a similar purpose of avoiding deeply nested calls.

Flipping arguments. `flip` takes a curried function of two arguments and returns a version with the arguments swapped:

```
flip :: (a -> b -> c) -> b -> a -> c
flip f y x = f x y
```

`flip` is particularly useful when we want to partially apply a function on its *second* argument. For instance, `flip (++) "!"` is a function that appends `"!"` to any string — without `flip`, we would need a lambda.

Point-free style. Combining partial application, sections, and composition, it is possible to define many functions without ever naming their arguments. This is called *point-free* style (the "points" being the arguments). Compare:

```
-- with explicit arguments
doubleAll xs = map (* 2) xs
keepEvens xs = filter even xs
sumSquares xs = sum (map (^ 2) xs)

-- point-free
doubleAll  = map (* 2)
keepEvens  = filter even
sumSquares = sum . map (^ 2)
```

In the point-free versions, the argument `xs` disappears: each function is defined as a composition or partial application of other functions, without ever mentioning the list it operates on. The idea is that `map (* 2)` already *is* a function from lists to lists — there is no need to name its argument just to pass it through.

Point-free style is natural in Haskell because currying, sections, and composition work together seamlessly. In Scala, the same concepts exist but the style is less idiomatic: partial application requires explicit curried signatures, there are no sections, and the `_` placeholder only works inline. Scala programmers typically favour explicit lambda expressions where Haskell programmers would write point-free definitions.

Point-free definitions are often more readable for simple pipelines — `sum . map (^ 2)` clearly says "square each element, then sum" — but can become obscure for complex expressions. The key is to use point-free style when it clarifies, and to fall back to explicit arguments when it doesn't.

Functions vs closures

Functions are *first class* values, and *function literals* (lambdas) are the device that functional languages make available to denote such values.

So far, all the examples of functions and function literals have been introduced as *closed* expressions, referring only to their parameters and no free variable. As in the lambda calculus, however, functions

may have free variables, bound by the enclosing context. Here is a simple example:

```haskell
addx :: Int -> Int -> Int
addx x = \y -> x + y
```

Within the lambda `\y -> x + y`, variable `x` is free, and its binding is provided by the enclosing context. The result is a **closure**: a pair carrying the code of the function and a link to the environment where its free variables are bound.

Two questions are of interest with respect to closures.

Scope. In languages with *lexical* (static) scope — like Haskell, Scala, and Elixir — the bindings that close the free variables of a closure are determined by the context of the closure's *definition site*, not its *call site*. To illustrate:

```haskell
showScope :: Int
showScope =
  let x = 3
      f y = x + y
      x' = 4          -- a different binding, not related to f's x
  in f 2 + x'
-- result: 9   (f 2 = 3+2 = 5, plus x' = 4)
```

The `x` inside `f` is bound to `3` (the definition context), not to any later rebinding. This is the essence of lexical scoping — and it is shared by all three languages of the course.

The same example in Scala:

```scala
def showScope: Int =
  val x = 3
  def f(y: Int) = x + y
  val x2 = 4
  f(2) + x2
// result: 9
```

Mutability. In Haskell, the question of what happens if a captured variable is mutated after the closure is created simply does not arise: there are no mutable variables. In Elixir, rebinding a variable creates a *new* binding — the closure retains the value at capture time, unaffected by later rebindings. The question is only interesting in languages with mutable state.

In Scala, closures capture *variable references*, not values. If a captured `var` is later mutated, the closure sees the new value:

```
var more = 1
val addMore = (x: Int) => x + more
addMore(10)  // 11

more = 37
addMore(10)  // 47
```

This can be a source of subtle bugs. Java avoids the issue entirely by requiring that variables captured by a lambda be `final` or *effectively final*. The functional programmer's recommendation is clear: avoid mutable state, and the question disappears.

A note on polymorphism

As the various examples of this and the previous lecture show, higher-order functions and polymorphic types go naturally together. Every function we have defined — `map`, `foldr`, `filter`, `curry`, `compose`, `flip` — has type variables in its signature. This is not a coincidence: because these functions capture *general* processing patterns, they must work for any type, and polymorphism is what makes that possible.

But there is more to the story than mere generality. The polymorphism of these functions has deep consequences — it constrains what the functions *can* do, it enables powerful reasoning principles, and it interacts with type inference in ways that differ significantly across languages. We take up these questions in the next lecture, where we study *parametric polymorphism* and the remarkable principle of *parametricity*.

Exercises

Exercise 1. Design the following functions (give types and definitions):

- `twice` that takes a function `f :: a -> a` and returns a function that applies `f` twice.
- `thrice` that applies `f` three times.
- `once` and `zero` that apply `f` one and zero times, respectively. (Make sure you have a clear idea of what it means to apply a function zero times.)

Exercise 2. The functions in the previous exercise look curiously similar to numbers: all they do is repeat their input some number of times. Let's call such functions *repeaters*. A repeater takes a unary

function `f` and outputs a function that repeatedly applies `f` to its argument. The type:

```
type Repeater a = (a -> a) -> (a -> a)
```

Design a function `rep2nat :: Repeater Int -> Int` which consumes a repeater and produces the number of times it repeats its argument, so that:

```
rep2nat zero   == 0
rep2nat once   == 1
rep2nat twice  == 2
rep2nat thrice == 3
```

Exercise 3. Write the function `repSucc :: Repeater a -> Repeater a` that increments a repeater by 1.

Exercise 4. Write the function `nat2rep :: Int -> Repeater a` that converts a natural number `n` into the repeater that repeats its input function `n` times. Try to define it without using `zero` or `repSucc` — just lambdas.

Exercise 5. Write the functions `repAdd` and `repMul` that take two repeaters and return the repeater representing their sum and product, respectively. Do not use `rep2nat`, `nat2rep`, or any built-in arithmetic.

REFERENCES

The material of this lecture draws on Chapter 12 of *Programming in Haskell (2nd Edition)*, G. Hutton, Cambridge University Press, 2016, and Chapter 15 of *Programming in Scala (5th Edition)*, M. Odersky, L. Spoon, B. Venners, F. Sommers, Artima Press, 2021.